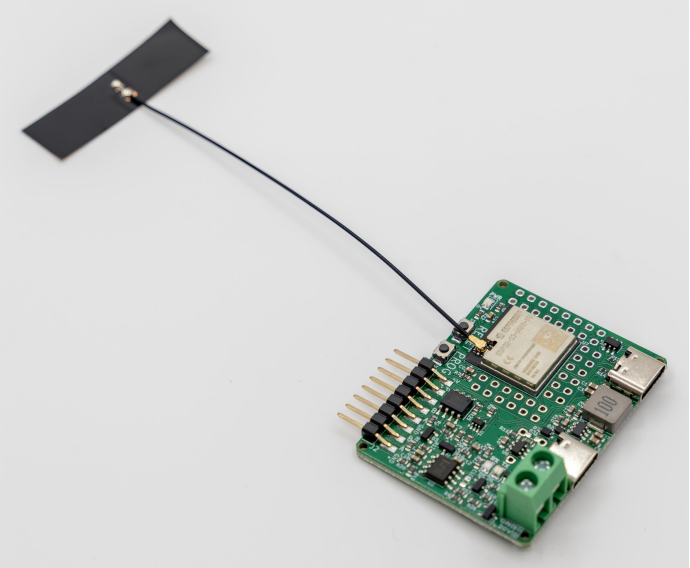
PROGRAMMABLE MOTION CONTROL

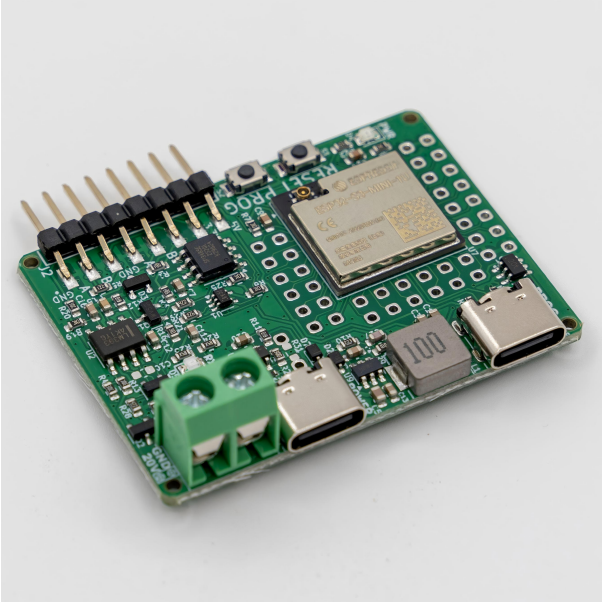
RS485 Controller

ESP32-S3 development board for Gearotons servomotors and other RS485 devices

Affordable and Simple All-in-One Motion Control

From Education to Innovation





Actual product photographs. External antenna included; cables, power supply and motors are separate system components.

11.7 g
Including attached antenna

230400 baud
Canonical example setting

PCB 0.5
Hardware covered here

A compact controller for shared-bus motion

Run motor-control logic directly on the ESP32-S3. The board combines native USB programming, a half-duplex RS485 interface, a USB-C PD power input and accessible GPIO pads in a compact open PCB.

Embedded control: dual-core LX7 processor, up to 240 MHz; 2.4 GHz Wi-Fi and Bluetooth 5 LE capabilities depend on application firmware.

Simple wiring: two parallel A/B/GND groups, automatic RS485 direction circuitry, reset and programming buttons.

Motor-power integration: standard assembly requests 20 V USB-C PD, with a direct motor-power output at the screw terminal.

SPECIFICATIONS

Electrical & power

Design values are distinguished from measurements and qualified product limits.

Parameter	Specification / conditions
Processor / module	ESP32-S3-MINI-1U; dual-core Xtensa LX7, up to 240 MHz.
Wireless hardware	2.4 GHz Wi-Fi 802.11b/g/n and Bluetooth 5 LE; external antenna included. Wireless control requires suitable firmware.
Programming supply, J5	5 V nominal USB input. Native ESP32-S3 USB data connection; motor power is supplied separately.
PD power input, J3	Standard design requests 20 V using CH221K and R27 = 200 kΩ. Actual voltage depends on the source and successful PD negotiation.
Motor-power output, J1	Pin 1 = J3 VBUS rail (labelled +20 V); pin 2 = GND. Direct pass-through, not a separately regulated 20 V output.
Logic / RS485 interface	3.3 V logic; SP3485EN half-duplex differential transceiver. Controller, USB and motor grounds are common; no galvanic isolation.
RS485 example setting	230400 baud, 8 data bits, no parity, 1 stop bit. TX = GPIO4; RX = GPIO5. Board maximum baud rate is not specified.
Bus loading / protection	R5 = 220 Ω across A/B. R1 = 560 Ω pulls A high; R3 = 560 Ω pulls B low. SM712 transient suppressor on the bus.
Auxiliary power pins, J2	3.3 V rail and J5-associated 5 V rail. Available external current is not specified. The 5 V rail is not generated from the PD input.

Verify motor power before connecting

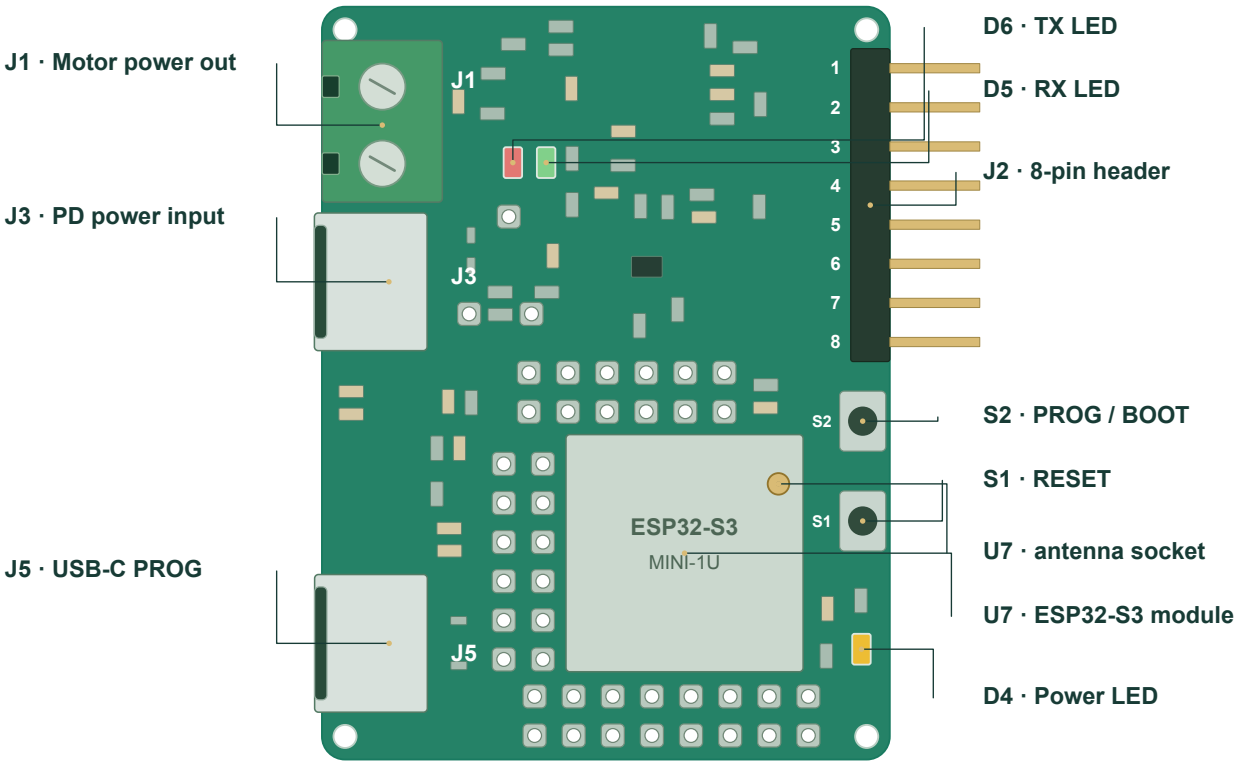
Use a USB-C PD supply that explicitly offers 20 V and a suitable cable. Measure J1 before attaching motors; a non-PD or unsuccessful source may leave the rail at a lower voltage. J5 alone does not provide motor power at J1.

Not yet specified: board current consumption, continuous motor-power pass-through current, maximum cable length and environmental limits. Component ratings are not board ratings; product-level ESD/EMC ratings have not been established.

INTERFACES · PHYSICAL LOCATOR

Power, programming & status

Locate connectors, switches and indicator LEDs before connecting or programming the controller.



Component side (B.Cu) · J2 pin 1 at top · connectors extend beyond PCB

The locator covers J1/J2/J3/J5, S1/S2, D4/D5/D6 and the U7-integrated antenna coax socket. Attach the included antenna to that socket before using the radio. References match the PCB and the electrical table on the next page; observe the stated viewing side.

Separate the two USB-C functions

J3: USB-C PD motor-power input. **J5:** native USB programming and 5 V logic power. The J3 USB data pins are unconnected. USB-C connector shape does not indicate which function it provides.

INTERFACES · ELECTRICAL REFERENCE

Connector pinout & controls

The two RS485 groups on J2 are electrically parallel; all grounds share one reference.

J2 pin	Signal	Use
1	GND	Common ground
2	A	RS485 A
3	B	RS485 B
4	GND	Same net as pin 1
5	A	Same bus as pin 2
6	B	Same bus as pin 3
7	3.3 V	Logic supply rail; external load not rated
8	5 V	Programming USB supply rail; external load not rated

Power, programming, switches and indicators

Reference	Function
J1 · POWER OUT	Two-way screw terminal. Pin 1 is the positive PD rail; pin 2 is GND. Verify polarity on the board.
J3 · PD POWER IN	USB-C power input and PD negotiation. This socket does not program the ESP32-S3.
J5 · USB-C PROG	Native USB D–/D+ connect to GPIO19/GPIO20. Use a data-capable USB cable.
S1 · RESET	Pulls the module EN signal low to reset the controller.
S2 · PROG	Pulls GPIO0 low; hold during reset to enter ROM download mode.
D4 · yellow LED	Logic-power indication.
D5 · green LED	Receive activity; indication depends on bus traffic.
D6 · red LED	Transmit activity; indication depends on bus traffic.

Keep motor supply voltage off J2. Its 3.3 V and 5 V pins are auxiliary logic rails, not motor-power terminals. Refer to the physical locator and actual PCB labels for orientation.

SYSTEM WIRING

Connect two servomotors

One 20 V-capable USB-C PD source can feed J3 and supply the motors through J1.

Use the following net-by-net connections for both motors. Connect the data bus along one route and distribute motor supply/return wiring for the combined motor current.

Net	Controller connection	Motor 1 and Motor 2
+20 V	J1 pin 1	Each motor V+ / positive supply
GND	J1 pin 2 and J2 pin 1 or 4	Each motor GND / supply return
A	J2 pin 2 or 5	A to A, onward along the bus
B	J2 pin 3 or 6	B to B, onward along the bus

1. Check power while disconnected. With motors disconnected, connect a 20 V-capable PD source to J3 and measure J1. Check polarity, then turn the source off before wiring the motors.

2. Wire the common ground and motor supply. Connect both motor supply returns and controller GND. Motor returns must be sized for motor current; the data-ground lead must not be the only motor-current return.

3. Connect the differential bus. Connect A to A and B to B. Route A/B as a twisted pair along one bus, keeping branches short. Give each motor a distinct alias before addressed motion commands.

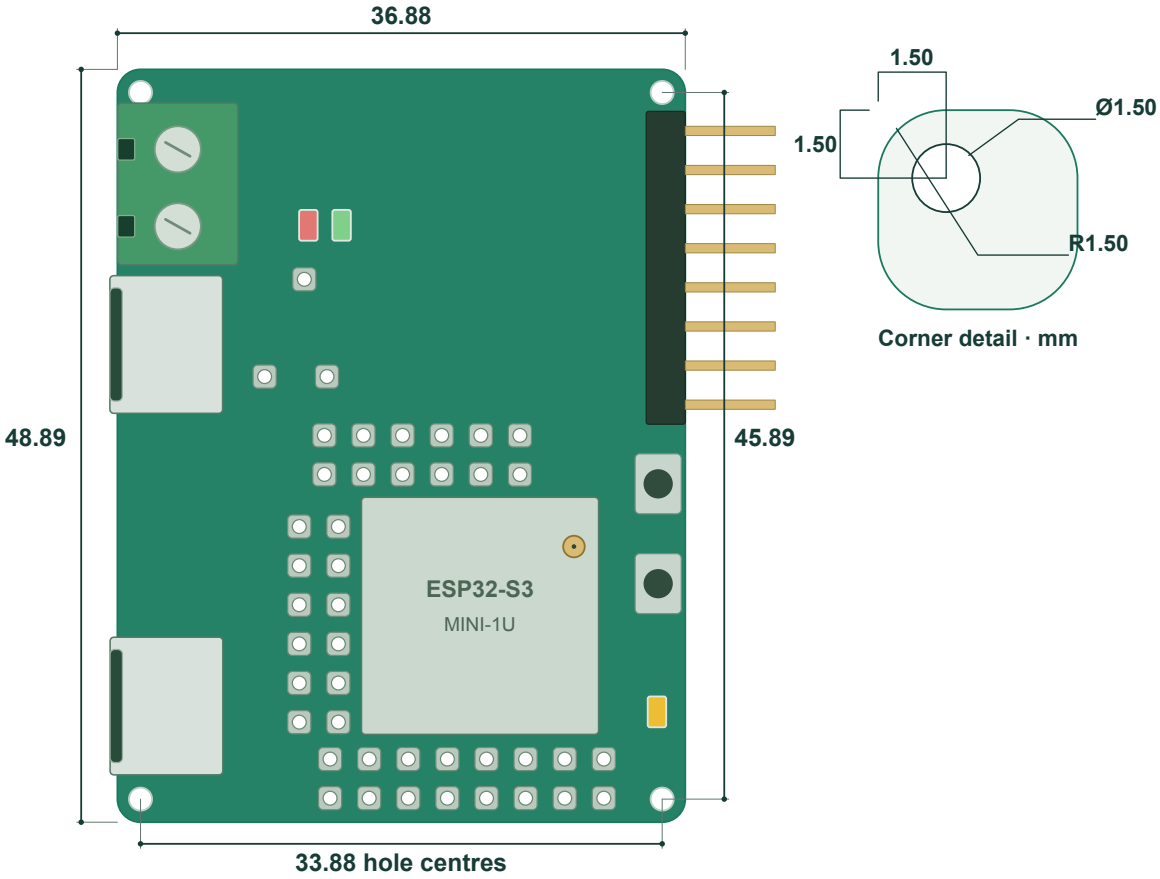
4. Power and test deliberately. Secure both motors before discovery or movement examples. The power supply, cable and distribution wiring must support the combined load.

The M17 motor reference specifies 12–24 V operation; the controller standard PD request is 20 V. R5 already loads A/B with 220 Ω. Select additional end termination for the complete bus instead of adding resistors at every motor. No fixed motor-count guarantee is specified.

MECHANICAL INTEGRATION

Detailed PCB dimensions

Nominal revision 0.5 KiCad geometry. Dimensions are in millimetres and exclude connector overhang and antenna.



4 × Ø1.50 holes · centres 1.50 from each adjacent edge · corner R1.50

Use the dimensioned drawing for board outline, corner radius, mounting-hole diameter, hole-centre spacing and edge insets. These are design values without manufacturing tolerances; confirm enclosure clearances on a physical unit.

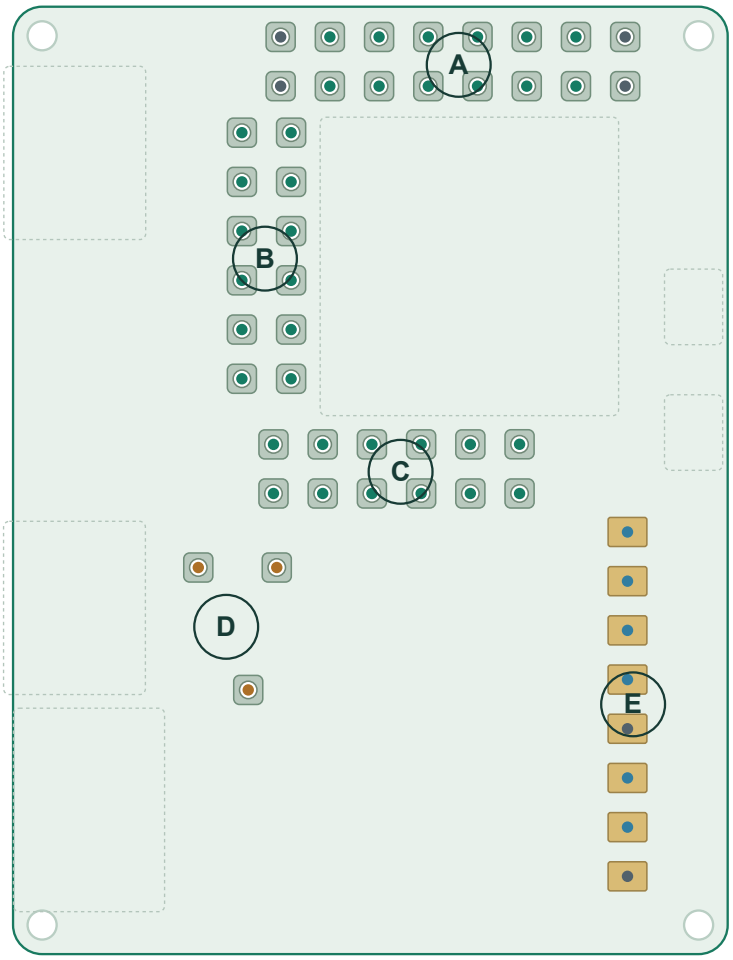
Mounting and assembled envelope

PCB thickness is 1.6 mm nominal. Keep clearance around underside components and connector solder joints. Overall assembled height and antenna/cable envelope are not specified. The Ø1.50 mm holes are not M2 or M3 clearance holes.

GPIO ACCESS · BOARD OVERVIEW

Find every expansion pad

Use the full-PCB pad-position view before consulting the enlarged pad-by-pad regions on the following pages.



Label side (F.Cu) · coloured marks locate all 51 test points

A–D: through-hole pads · E: surface service pads

The map locates 43 through-hole pads: 36 GPIO-linked, four GND and three PD-control points. TP43 (PG), TP44 (VDD) and TP47 (CFG) belong to the PD circuit and are not GPIO. Use the stated PCB side and the enlarged regions to identify each physical pad.

3.3 V logic only

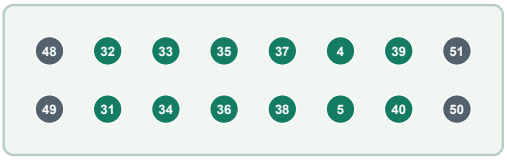
GPIO4/GPIO5 are used by the onboard RS485 path. GPIO19/GPIO20 carry native USB. GPIO0/3/45/46 are ESP32-S3 strapping signals; GPIO0 is connected to PROG. External circuitry must not disturb boot, USB or the RS485 interface.

GPIO ACCESS · ADDITIONAL ENLARGEMENT

Pad region A

Additional enlarged region from the same verified PCB geometry; every pad retains its individual label.

Region A · label side (F.Cu)



TP4
GPIO43

TP5
GPIO44

TP31
GPIO35

TP32
GPIO36

TP33
GPIO37

TP34
GPIO38

TP35
GPIO39

TP36
GPIO40

TP37
GPIO41

TP38
GPIO42

TP39
GPIO45

TP40
GPIO46

TP48
GND

TP49
GND

TP50
GND

TP51
GND

Markers show TP reference numbers; the matching labels identify each signal.

Refer to the full-PCB overview for region location and viewing side. Each marker carries its TP reference number; the matching TP entry identifies that pad's GPIO, ground or PD/service net. TP reference numbers are not GPIO numbers.

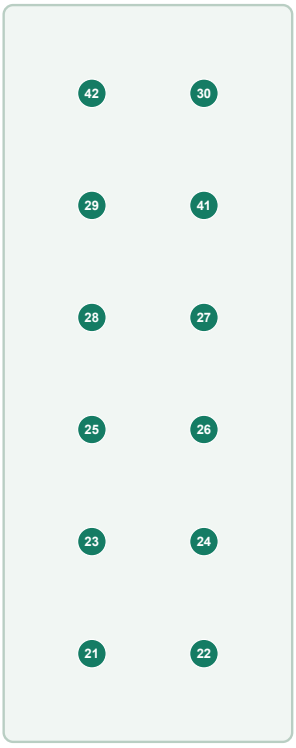
Module pin definitions: Official Espressif ESP32-S3-MINI-1 / MINI-1U datasheet (PDF).

GPIO ACCESS · ADDITIONAL ENLARGEMENT

Pad region B

Additional enlarged region from the same verified PCB geometry; every pad retains its individual label.

Region B · label side (F.Cu)



TP21
GPIO13

TP22
GPIO14

TP23
GPIO15

TP24
GPIO16

TP25
GPIO17

TP26
GPIO18

TP27
GPIO21

TP28
GPIO26

TP29
GPIO33

TP30
GPIO34

TP41
GPIO47

TP42
GPIO48

Markers show TP reference numbers; the matching labels identify each signal.

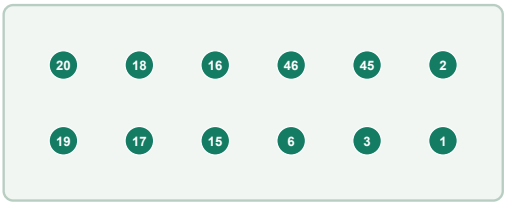
Refer to the full-PCB overview for region location and viewing side. Each marker carries its TP reference number; the matching TP entry identifies that pad’s GPIO, ground or PD/service net. TP reference numbers are not GPIO numbers.

GPIO ACCESS · ADDITIONAL ENLARGEMENT

Pad region C

Additional enlarged region from the same verified PCB geometry; every pad retains its individual label.

Region C · label side (F.Cu)



TP1
GPIO1

TP2
GPIO2

TP3
GPIO3

TP6
GPIO6

TP15
GPIO7

TP16
GPIO8

TP17
GPIO9

TP18
GPIO10

TP19
GPIO11

TP20
GPIO12

TP45
GPIO4

TP46
GPIO5

Markers show TP reference numbers; the matching labels identify each signal.

Shared RS485 signals: TP45 is GPIO4 / TX and TP46 is GPIO5 / RX. These pads share the active RS485 interface and are not uncommitted GPIO.

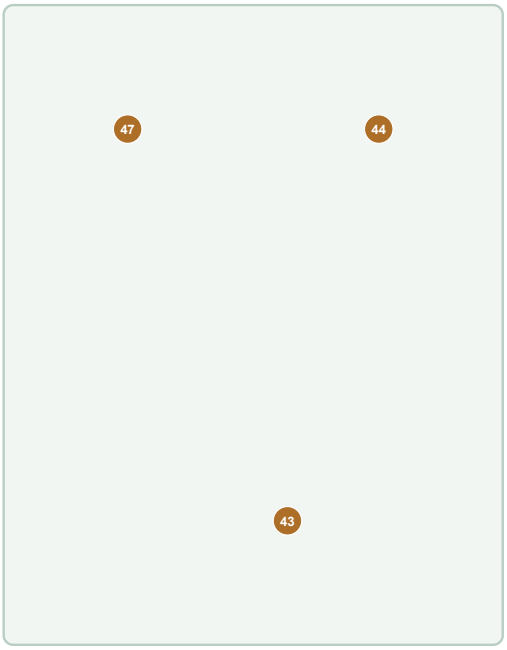
Refer to the full-PCB overview for region location and viewing side. Each marker carries its TP reference number; the matching TP entry identifies that pad's GPIO, ground or PD/service net. TP reference numbers are not GPIO numbers.

GPIO ACCESS · ADDITIONAL ENLARGEMENT

Pad region D

Additional enlarged region from the same verified PCB geometry; every pad retains its individual label.

Region D · label side (F.Cu)



TP43
PD_PG

TP44
PD_VDD

TP47
PD_CFG

Markers show TP reference numbers; the matching labels identify each signal.

Refer to the full-PCB overview for region location and viewing side. Each marker carries its TP reference number; the matching TP entry identifies that pad’s GPIO, ground or PD/service net. TP reference numbers are not GPIO numbers.

GPIO ACCESS · ADDITIONAL ENLARGEMENT

Pad region E

Additional enlarged region from the same verified PCB geometry; every pad retains its individual label.

Region E · label side (F.Cu)



TP7
GND

TP8
RS485 A

TP9
RS485 B

TP10
GND

TP11
RS485 A

TP12
RS485 B

TP13
+3.3V

TP14
+5V

Markers show TP reference numbers; the matching labels identify each signal.

Surface-mount service pads; these are not through holes.

Refer to the full-PCB overview for region location and viewing side. Each marker carries its TP reference number; the matching TP entry identifies that pad’s GPIO, ground or PD/service net. TP reference numbers are not GPIO numbers.

GPIO ACCESS · CHECKED NET LIST

Expansion pad net reference

All 51 TP references: 43 through-hole pads and eight SMD service points. “SMD” marks a surface pad, not a GPIO hole.

TP	Net / shared function
TP1	GPIO1
TP2	GPIO2
TP3	GPIO3
TP4	GPIO43 / UART0 TX
TP5	GPIO44 / UART0 RX
TP6	GPIO6
TP7	GND [SMD]
TP8	RS485 A [SMD]
TP9	RS485 B [SMD]
TP10	GND [SMD]
TP11	RS485 A [SMD]
TP12	RS485 B [SMD]
TP13	+3.3V [SMD]
TP14	+5V [SMD]
TP15	GPIO7
TP16	GPIO8
TP17	GPIO9
TP18	GPIO10
TP19	GPIO11
TP20	GPIO12
TP21	GPIO13
TP22	GPIO14
TP23	GPIO15
TP24	GPIO16
TP25	GPIO17
TP26	GPIO18

TP	Net / shared function
TP27	GPIO21
TP28	GPIO26
TP29	GPIO33
TP30	GPIO34
TP31	GPIO35
TP32	GPIO36
TP33	GPIO37
TP34	GPIO38
TP35	GPIO39
TP36	GPIO40
TP37	GPIO41
TP38	GPIO42
TP39	GPIO45
TP40	GPIO46
TP41	GPIO47
TP42	GPIO48
TP43	PD_PG
TP44	PD_VDD
TP45	GPIO4 / RS485 TX
TP46	GPIO5 / RS485 RX
TP47	PD_CFG
TP48	GND
TP49	GND
TP50	GND
TP51	GND

BRING-UP

Getting started

Prepare the hardware and software, establish communication, then enable motion deliberately.

1. **Prepare the hardware.** Attach the included external antenna. With power off, check the motor polarity and A/B/GND connections. Secure the mechanism before running any movement example.

2. **Use the programming port.** Connect J5 USB-C PROG with a data-capable cable. J3 is the separate USB-C PD power input, not a programming port.

3. **Install Arduino support.** Install the Arduino ESP32 board package and “Servomotor by Gearotons” from Library Manager. Select an ESP32-S3 target; configure native USB CDC for the USB serial console as appropriate for the board package.

4. **Enter download mode if needed.** Hold S2 PROG, press and release S1 RESET, then release PROG. Select the detected USB device and upload the application.

5. **Verify motor power and identity.** Check J1 voltage before connecting motors. Use the correct motor alias and ensure the mechanism can safely perform one full rotation before running either canonical example.

Two unchanged M17 reference programs

The following pages reproduce the Python and Arduino “Basic Motor Control” programs from the latest English M17 servomotor datasheet, version 1.10. The code is copied verbatim; setup notes are kept outside the code.

Python runs on a host computer with an appropriate RS485 serial interface. J5 is native ESP32-S3 USB; it is not automatically a transparent USB-to-RS485 adapter. A USB/RS485 bridge would require firmware providing that behavior.

Both examples command motor motion

Each example enables the motor drive and requests one shaft rotation in one second, then disables the drive. Match the alias and serial connection to the intended motor. The unmodified reference code is shown for consistency with the M17 documentation.

GETTING STARTED · CANONICAL PYTHON

Python: basic motor control

Verbatim program from the latest English M17 servomotor datasheet, v1.10 · reference PDF page 11.

Install on the host: `pip3 install servomotor`. The example retains its original serial-port setting, alias, comments and API calls.

```
#!/usr/bin/env python3
"""
Minimal trapezoid move: rotate 1 turn in 1 second.
Edit ALIAS below if needed. Uses rotations and seconds.
"""
import time, servomotor
from servomotor import communication

# Hard-coded settings for a minimal demo
ALIAS = 'X'                             # Device alias, change if needed
SERIAL_PORT = "/dev/tty.usbserial-110"  # Serial device path; change if needed (e.g., "COM3" on
                                        #  Windows)
DISPLACEMENT_ROTATIONS = 1.0            # 1 rotation
DURATION_SECONDS = 1.0                  # 1 second
DELAY_MARGIN = 0.10                     # +10% wait margin because the motor's clock is not
                                        #  perfectly accurate

communication.serial_port = SERIAL_PORT # if you comment this out then the program
                                        #  should prompt you for the serial port or it will use
                                        #  the last used port from a file
servomotor.open_serial_port()

m = servomotor.M3(ALIAS, time_unit="seconds", position_unit="shaft_rotations", verbose=0)
m.enable_mosfets()
m.trapezoid_move(DISPLACEMENT_ROTATIONS, DURATION_SECONDS)
time.sleep(DURATION_SECONDS * (1.0 + DELAY_MARGIN))
m.disable_mosfets()

servomotor.close_serial_port()
```

Canonical source: `python_programs/example_trapezoid_move.py`, recovered from the source revision corresponding to the published M17 example. No controller-specific edits have been made to this listing.

Python API: github.com/tomrodinger/servomotor/tree/main/python_programs

GETTING STARTED · CANONICAL ARDUINO

Arduino: basic motor control

Verbatim program from the latest English M17 servomotor datasheet, v1.10 · reference PDF page 13.

Install “Servomotor by Gearotons” using Arduino Library Manager. The unchanged ESP32 pin definitions in this reference are GPIO4 TX and GPIO5 RX.

```
// Minimal Arduino example: Trapezoid move using built-in unit conversions
// Goal: spin the motor exactly 1 rotation in 1 second, then stop.
// Sequence:
//  enable MOSFETs -> trapezoidMove(1.0 rotations, 1.0 seconds) -> wait 1.1s -> disable MOSFETs.
//
// Notes:
// - This uses the library's unit conversion (no raw counts/timesteps).
// - Configure Serial1 pins for your board (ESP32 example pins below).
// - Motor is created AFTER Serial1.begin(...) so hardware UART pins are set first.

#include <Servomotor.h>

#define ALIAS 'X'                   // Device alias
#define BAUD 230400                 // RS485 UART baud rate
#define DISPLACEMENT_ROTATIONS 1.0f // 1 rotation
#define DURATION_SECONDS 1.0f       // 1 second
#define TOLERANCE_PERCENT 10        // +10% wait margin because the motor's clock is not
                                    //  perfectly accurate
#define WAIT_MS ((unsigned long)(DURATION_SECONDS * 1000.0f * (100 + TOLERANCE_PERCENT) / 100))

// Example RS485 pins for ESP32 DevKit (change as needed for your board)
#if defined(ESP32)
#define RS485_TXD 4             // TX pin to RS485 transceiver
#define RS485_RXD 5             // RX pin from RS485 transceiver
#endif

void setup() {
  Serial.begin(115200); // Console serial for debugging

  // Create the motor; serial port opens on first instantiation.
#if defined(ESP32)
  Servomotor motor(ALIAS, Serial1, RS485_RXD, RS485_TXD);
#else
  Servomotor motor(ALIAS, Serial1);
#endif

  // Use units: rotations for position, seconds for time
  motor.setPositionUnit(PositionUnit::SHAFT_ROTATIONS);
  motor.setTimeUnit(TimeUnit::SECONDS);

  motor.enableMosfets();
  motor.trapezoidMove(DISPLACEMENT_ROTATIONS, DURATION_SECONDS);
  delay(WAIT_MS);
  motor.disableMosfets();
}

void loop() {
}
```

This is the program printed in M17 datasheet v1.10, not a newer working-tree variant. Source snapshot and PDF correspondence are recorded in the accompanying example-provenance JSON.

DESIGN REFERENCE

Controller schematic & resources

The appendix reproduces the full controller sheet with ESP32-S3-MINI-1U retained as one module block.

Full controller schematic

The following landscape appendix embeds the supplied revision 0.5 controller schematic as vector content. It includes the board power, PD negotiation, native USB, RS485, controls and indicator circuitry. Zoom in to read component values without a raster-image resolution limit.

Source sheet: `PCB/0.5/schematic/0.5-index-schTop.pdf`. The module symbol and its external connections remain intact; no ESP32 module-internal schematic has been added.

Official module documentation

Espressif ESP32-S3-MINI-1 / ESP32-S3-MINI-1U datasheet
Manufacturer reference for the module pin definitions, boot configuration, radio, electrical specifications and module mechanical details.

Software, tutorials and design files

Controller source: github.com/tomrodinger/RS485_Controller
Motor setup and protocol: tutorial.gearotons.com
Arduino library: github.com/tomrodinger/Servomotor_Arduino_Library
Feedback: tutorial.gearotons.com/feedback

Specification and review basis

PCB 0.5 schematic, board geometry and assembly BOM; supplied firmware examples; owner-provided 11.7 g mass and included-antenna statement; M17 English datasheet v1.10; official Espressif, MaxLinear and TI component references. No component current, environmental or ESD rating has been promoted to a board-level guarantee.

DS v0.2 · owner review draft

Prepared 27 October 2026. Physical product measurements and limits remain subject to confirmation where marked not specified. Hardware/software source is supplied under the repository MIT licence. The v0.1 review PDF is preserved separately.

Full controller schematic

PCB 0.5 · module retained as a black box · A3 landscape appendix for comfortable digital zoom and larger-format printing.

